LAPORAN PRAKTIKUM PEKAN 3

STRUKTUR DATA

Disusun oleh:

FAHRI SYAH PUTRA RAMADHAN

2511531015

Dosen pengampu:

Dr. Wahyudi. MT

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG, 2026

PENJELASAN KODE PROGRAM NILAI MAKSIMUM

Program NilaiMaksimum_2511531015 dibuat untuk mencari **nilai maksimum** dari sekumpulan data yang disimpan dalam struktur data **Stack** pada Java. Stack adalah struktur data yang bekerja dengan prinsip **LIFO (Last In First Out)**, yaitu data yang terakhir dimasukkan akan menjadi data pertama yang diambil.

Pada program ini, digunakan Stack<Integer> sehingga data yang disimpan berupa bilangan bulat. Class Stack diimpor dari package java.util.Stack. Program terdiri dari dua bagian utama, yaitu method max() untuk mencari nilai terbesar dan method main() untuk menjalankan program.

Fungsi Setiap Bagian Program

1. Package dan Import

packagePekan3_2511531015;
import java.util.Stack;

Bagian ini menunjukkan bahwa class berada pada package Pekan3_2511531015 dan menggunakan library Stack dari Java.

2. Class NilaiMaksimum_2511531015

public class NilaiMaksimum_2511531015

Class ini merupakan class utama yang berisi method untuk mencari nilai maksimum pada stack.

3. Method max(Stack<Integer> s)

Method ini dibuat untuk mencari nilai terbesar dari elemen-elemen dalam stack s.

Alur kerja method max():

- Program membuat stack baru bernama backup sebagai tempat penyimpanan sementara.
- Elemen paling atas dari stack s diambil dengan pop() lalu disimpan sebagai nilai awal maksimum.
- Nilai tersebut juga dimasukkan ke stack backup.
- Selanjutnya, elemen-elemen lain dalam stack dibandingkan satu per satu dengan nilai maksimum saat ini.
- Jika ditemukan nilai yang lebih besar, maka maxValue akan diperbarui.
- Setelah semua elemen selesai diperiksa, isi backup dikembalikan lagi ke stack awal agar data tidak hilang.

Jadi, method ini tidak hanya mencari nilai terbesar, tetapi juga berusaha **menjaga isi stack tetap seperti semula** setelah proses selesai.

4. Method main()

Pada method main(), program membuat sebuah stack lalu mengisi data:

s.push(70);
s.push(12);
s.push(20);

Artinya, stack berisi tiga nilai, yaitu 70, 12, dan 20. Setelah itu program menampilkan:

- isi stack,
- elemen teratas stack dengan peek(),
- dan hasil nilai maksimum dengan memanggil max(s).

Jika logika program berjalan dengan benar, maka nilai maksimum dari stack tersebut adalah **70**.

Maksud Penggunaan push(), pop(), dan peek()

Dalam program ini terdapat beberapa operasi stack, yaitu:

- push() digunakan untuk menambahkan elemen ke stack.
- pop() digunakan untuk mengambil sekaligus menghapus elemen teratas stack.
- peek() digunakan untuk melihat elemen teratas stack tanpa menghapusnya.

Operasi-operasi ini menunjukkan cara kerja dasar struktur data stack dalam Java.

PENJELASAN KODE PROGRAM SISWA STACK

Program di atas dibuat untuk menerapkan konsep **stack** menggunakan ArrayList di Java. Data yang disimpan dalam stack bukan bilangan biasa, tetapi berupa **objek siswa** yang memiliki atribut **nama** dan **nim**. Dengan demikian, program ini menunjukkan bahwa stack tidak hanya dapat digunakan untuk tipe data sederhana, tetapi juga untuk menyimpan objek.

Secara umum, program terdiri dari dua class, yaitu:

1. Siswa_2511531015, yang berfungsi sebagai class untuk merepresentasikan data siswa.
2. SiswaStack_2511531015, yang berfungsi sebagai stack untuk menyimpan objek-objek siswa.

1. Class Siswa_2511531015

Class ini digunakan untuk menyimpan data seorang siswa. Di dalamnya terdapat dua atribut, yaitu:

- String nama → untuk menyimpan nama siswa
- int nim → untuk menyimpan nomor induk siswa

Constructor

```
public Siswa_2511531015(String nama, int nim) {
    this.nama = nama;
    this.nim = nim;
}
```

Constructor ini digunakan untuk mengisi nilai nama dan nim saat objek siswa dibuat. Kata this digunakan untuk membedakan atribut class dengan parameter yang dikirimkan.

Method toString()

```
@Override
public String toString() {
    return "Nim: " + nim + ", Nama: " + nama;
}
```

Method toString() berfungsi untuk menampilkan data siswa dalam bentuk teks. Dengan method ini, saat objek siswa dicetak menggunakan System.out.println(), yang tampil adalah isi datanya, bukan alamat objek di memori.

Contohnya:

Nim: 1, Nama: Ali

2. Class SiswaStack_2511531015

Class ini digunakan untuk membuat stack siswa. Stack dibuat dengan memanfaatkan ArrayList sebagai tempat penyimpanan data.

Deklarasi stack

```
private ArrayList<Siswa_2511531015> stack;
```

Variabel stack digunakan untuk menyimpan objek-objek siswa. Karena menggunakan ArrayList, maka elemen dapat ditambahkan dan dihapus dengan mudah.

Constructor

```
public SiswaStack_2511531015() {
    stack = new ArrayList<>();
}
```

Constructor ini digunakan untuk membuat objek ArrayList baru saat stack pertama kali dibuat.

3. Method pada Stack

a. Method push_1015()

```
public void push_1015(Siswa_2511531015 mhs) {
    stack.add(mhs);
}
```

Method ini digunakan untuk **menambahkan siswa ke dalam stack**. Karena konsep stack adalah **LIFO (Last In First Out)**, data yang terakhir dimasukkan akan berada di posisi paling atas.

Misalnya:

- push Ali
- push Boby
- push Charles

Maka urutan stack dari bawah ke atas adalah:
Ali → Boby → Charles

b. Method pop()

```
public Siswa_2511531015 pop() {
    if (!isEmpty()) {
        return stack.remove(stack.size() - 1);
    }
    return null;
}
```

Method pop() digunakan untuk **mengambil sekaligus menghapus** elemen paling atas dari stack. Jika stack tidak kosong, maka elemen terakhir pada ArrayList akan dihapus dan dikembalikan. Jika stack kosong, method akan mengembalikan null.

Karena elemen terakhir adalah elemen teratas stack, maka operasi ini sesuai dengan prinsip stack.

c. Method peek()

```
public Siswa_2511531015 peek() {
    if (!isEmpty()) {
        return stack.get(stack.size() - 1);
    }
    return null;
}
```

Method peek() digunakan untuk **melihat elemen paling atas** tanpa menghapusnya dari stack. Jadi method ini hanya menampilkan data siswa teratas, tetapi data tersebut tetap ada di dalam stack.

d. Method isEmpty()

```
public boolean isEmpty() {
    return stack.isEmpty();
}
```

Method ini digunakan untuk mengecek apakah stack kosong atau tidak.
Jika kosong, method akan mengembalikan nilai true, dan jika masih berisi data akan mengembalikan false.

e. Method tampilkanSiswaStack_2511531015()

```
public void tampilkanSiswaStack_2511531015() {
    for (int i = stack.size() - 1; i >= 0; i--) {
        System.out.println(stack.get(i));
    }
}
```

Method ini digunakan untuk menampilkan seluruh isi stack dari **atas ke bawah**. Perulangan dimulai dari indeks terakhir (stack.size() - 1) sampai indeks 0. Hal ini dilakukan karena elemen paling atas stack berada di posisi terakhir ArrayList.

Dengan demikian, tampilan stack sesuai dengan konsep sebenarnya, yaitu elemen teratas ditampilkan lebih dulu.

4. Penjelasan Method main()

Pada bagian main(), program mulai dijalankan.

Membuat objek stack

```
SiswaStack_2511531015 studentStack = new SiswaStack_2511531015();
```

Baris ini digunakan untuk membuat stack baru bernama studentStack.

Membuat objek siswa

```
Siswa_2511531015 mhs1 = new Siswa_2511531015("Ali", 1);
Siswa_2511531015 mhs2 = new Siswa_2511531015("Boby", 2);
Siswa_2511531015 mhs3 = new Siswa_2511531015("Charles", 3);
```

Program membuat tiga objek siswa, yaitu:

- Ali dengan NIM 1
- Boby dengan NIM 2
- Charles dengan NIM 3

Menambahkan siswa ke stack

```
studentStack.push_1015(mhs1);
studentStack.push_1015(mhs2);
studentStack.push_1015(mhs3);
```

Ketiga siswa dimasukkan ke dalam stack secara berurutan. Karena Charles dimasukkan terakhir, maka Charles akan menjadi elemen paling atas.

Menampilkan isi stack

System.out.println("Siswa di dalam stack:");
studentStack.tampilkanSiswaStack_2511531015();

Program menampilkan seluruh isi stack dari atas ke bawah.
Urutannya adalah:

1. Charles
2. Boby
3. Ali

Menampilkan siswa teratas

System.out.println("siswa teratas " + studentStack.peek());

Baris ini menampilkan siswa yang berada di posisi teratas, yaitu Charles.

Menghapus siswa teratas

System.out.println("mengeluarkan siswa teratas dari stack: " + studentStack.pop());

Baris ini menghapus siswa paling atas dari stack, yaitu Charles.

Menampilkan stack setelah pop

System.out.println("daftar siswa setelahdi pop:");
studentStack.tampilkanSiswaStack_2511531015();

Setelah Charles dihapus, isi stack yang tersisa adalah:

1. Boby
2. Ali

5. Konsep yang Dipelajari dari Program

Melalui program ini, dapat dipahami beberapa konsep penting, yaitu:

- penggunaan **class dan object** dalam Java,
- penggunaan **constructor** untuk menginisialisasi data,
- penggunaan method toString() untuk menampilkan objek,
- penerapan struktur data **stack**,
- serta penggunaan ArrayList sebagai media penyimpanan data stack.

PENJELASAN KODE PROGRAM STACK ARRAY

Program stackArray_2511531015 merupakan implementasi struktur data **stack** menggunakan **array** pada Java. Stack adalah struktur data yang bekerja dengan prinsip **LIFO (Last In First Out)**, yaitu data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan.

Pada program ini, stack dibuat secara manual tanpa menggunakan class Stack bawaan Java. Data disimpan dalam array bertipe int, sehingga elemen yang dimasukkan ke dalam stack adalah bilangan bulat

1. Deklarasi Class dan Variabel

```
public class stackArray_2511531015 {
    static final int MAX = 1000;
    int top;
    int a[] = new int[MAX];
```

Pada bagian ini terdapat tiga komponen utama:

- MAX = 1000
  Menunjukkan kapasitas maksimum stack, yaitu sebanyak 1000 elemen.
- int top
  Variabel top digunakan untuk menunjukkan posisi elemen paling atas pada stack.
- int a[] = new int[MAX]
  Array a digunakan sebagai tempat penyimpanan elemen-elemen stack.

2. Method isEmpty()

```
boolean isEmpty()
    {
        return (top < 0);
    }
```

Method ini digunakan untuk memeriksa apakah stack kosong atau tidak. Jika nilai top kurang dari 0, berarti stack belum memiliki elemen, sehingga method akan mengembalikan nilai true.

Method ini penting untuk menghindari kesalahan saat melakukan pengambilan data dari stack yang kosong.

3. Constructor stackArray_2511531015()

```
stackArray_2511531015()
        {
                top = -1;


        }
```

Constructor ini digunakan untuk menginisialisasi stack saat objek dibuat. Nilai awal top diatur menjadi -1, yang menandakan bahwa stack masih kosong.

Artinya, sebelum ada data yang dimasukkan, belum ada posisi yang ditempati dalam array.

4. Method push(int x)

```
if (top >= (MAX -1)) {
                        System.out.println("Stack Overflow");
                        return false;


                }
                else {
                        a[++top] = x;
                        System.out.println(x + " dimasukkan dalam Stack");
                        return true;
```

Method push() digunakan untuk **menambahkan elemen ke dalam stack**.

Cara kerjanya:

- Program terlebih dahulu memeriksa apakah stack sudah penuh.
- Jika top sudah mencapai MAX - 1, berarti stack tidak bisa menampung data lagi, sehingga muncul pesan **"Stack Overflow"**.
- Jika masih ada ruang, nilai top akan dinaikkan satu posisi terlebih dahulu dengan ++top, lalu data x disimpan ke array pada posisi tersebut.

Contohnya:

- Jika stack kosong, top = -1
- Saat data pertama dimasukkan, top menjadi 0
- Data disimpan pada a[0]

Method ini mengembalikan:

- true jika data berhasil dimasukkan
- false jika stack penuh

5. Method pop()

```
if(top < 0) {
                    System.out.println("Stack Underflow");
                    return 0;


          }else {
                    int x = a [top--];
                    return x;
```

Method pop() digunakan untuk **mengambil sekaligus menghapus elemen paling atas dari stack**.

Cara kerjanya:

- Program memeriksa apakah stack kosong.
- Jika kosong, akan muncul pesan **"Stack Underflow"** karena tidak ada data yang bisa diambil.
- Jika tidak kosong, elemen pada posisi top diambil dan disimpan ke variabel x.
- Setelah itu, nilai top dikurangi satu dengan top--, sehingga elemen tersebut dianggap sudah keluar dari stack.

Contoh:

- Jika top = 2, berarti elemen teratas ada di a[2]
- Setelah pop(), elemen a[2] diambil
- Nilai top menjadi 1

Method ini mengembalikan nilai elemen yang diambil.

6. Method peek()

```
{
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;

        }
        else {
            int x = a [top];
            return x;
        }
    }
```

Method peek() digunakan untuk **melihat elemen paling atas stack tanpa menghapusnya**.

Cara kerjanya:

- Jika stack kosong, program menampilkan pesan **"Stack Underflow"**
- Jika stack berisi data, method akan mengambil nilai pada indeks top

Berbeda dengan pop(), method peek() tidak mengurangi nilai top, sehingga data tetap tersimpan di dalam stack.

7. Method print()

```
for (int i = top; i>-1; i--) {
            System.out.print(" "+ a[i]);
```

Method print() digunakan untuk **menampilkan seluruh isi stack** dari atas ke bawah.

Cara kerjanya:

- Perulangan dimulai dari indeks top
- Kemudian terus menurun sampai indeks 0

Hal ini sesuai dengan konsep stack, karena elemen paling atas ditampilkan lebih dulu.

Sebagai contoh, jika data yang dimasukkan adalah:

- 10
- 20
- 30

Maka hasil print() adalah:

30 20 10

Karena 30 adalah elemen terakhir yang masuk dan berada di posisi paling atas.

8. Konsep yang Digunakan dalam Program

Program ini menerapkan beberapa konsep dasar dalam struktur data, yaitu:

a. Stack

Stack adalah struktur data linear yang hanya memperbolehkan operasi pada satu sisi, yaitu sisi atas (**top**).

b. LIFO

Prinsip kerja stack adalah **Last In First Out**, yaitu elemen terakhir yang masuk akan menjadi elemen pertama yang keluar.

c. Overflow dan Underflow

- **Overflow** terjadi ketika data dimasukkan saat stack sudah penuh.
- **Underflow** terjadi ketika data diambil atau dilihat saat stack kosong.

9. Kelebihan Program

Program ini cukup baik untuk memahami konsep dasar stack karena:

- menggunakan array sederhana,
- mudah dipahami,
- dan menunjukkan operasi dasar stack secara jelas, yaitu push, pop, peek, isEmpty, dan print.

Program ini juga membantu memahami bagaimana variabel top bekerja untuk mengatur posisi elemen teratas.

10. Kekurangan atau Catatan

Salah satu catatan pada program ini adalah saat terjadi kondisi error (Underflow), method pop() dan peek() mengembalikan nilai 0. Nilai ini bisa menimbulkan kebingungan jika 0 sebenarnya adalah data yang valid dalam stack. Dalam pengembangan program yang lebih lanjut, kondisi seperti ini biasanya ditangani dengan cara lain, misalnya menggunakan exception.

Selain itu, kode yang diberikan hanya berisi class stack, sehingga untuk menjalankannya masih perlu ditambahkan main() agar operasi stack bisa diuji secara langsung.

PENJELASAN KODE PROGRAM STACK ARRASY DRIVER

Program stackArrayDriver_2511531015 merupakan **class driver** yang digunakan untuk menjalankan dan menguji class stackArray_2511531015. Class driver ini berfungsi untuk membuktikan bahwa operasi-operasi dasar pada struktur data stack, seperti **push**, **pop**, **peek**, dan **print**, dapat berjalan dengan baik.

Program ini hanya berisi method main(), sehingga tugas utamanya adalah melakukan percobaan terhadap stack yang sudah dibuat pada class sebelumnya.

Fungsi Class stackArrayDriver_2511531015

Class ini berfungsi sebagai program utama untuk:

- membuat objek stack,
- memasukkan data ke dalam stack,
- mengeluarkan data dari stack,
- melihat elemen teratas,
- dan menampilkan isi stack.

Dengan kata lain, class ini digunakan sebagai sarana pengujian implementasi stack berbasis array.

Penjelasan Setiap Bagian Kode

1. Pembuatan Objek Stack

stackArray_2511531015 s = new stackArray_2511531015();

Baris ini digunakan untuk membuat sebuah objek stack bernama s dari class stackArray_2511531015. Setelah objek ini dibuat, maka program dapat menggunakan seluruh method yang ada pada class stack, seperti push(), pop(), peek(), dan print().

2. Menambahkan Data ke dalam Stack

s.push(10);
s.push(20);
s.push(30);

Ketiga baris ini digunakan untuk menambahkan data ke dalam stack. Data dimasukkan secara berurutan, yaitu:

- 10
- 20
- 30

Karena stack bekerja dengan prinsip **LIFO (Last In First Out)**, maka elemen yang terakhir dimasukkan, yaitu **30**, akan berada di posisi paling atas stack.

Setelah proses push, susunan stack dari bawah ke atas adalah:

- 10
- 20
- 30

3. Mengeluarkan Elemen Teratas

System.out.println(s.pop() + " dikeluarkan dari stackk");

Baris ini digunakan untuk mengambil sekaligus menghapus elemen yang berada di posisi paling atas stack.
Karena elemen teratas adalah **30**, maka nilai itulah yang akan dikeluarkan.

Hasil yang ditampilkan adalah:

30 dikeluarkan dari stackk

Setelah operasi pop(), isi stack berubah menjadi:

- 10
- 20

Artinya, sekarang elemen teratas stack adalah **20**.

4. Melihat Elemen Teratas

System.out.println("Elemen teratas adalah: " + s.peek());

Method peek() digunakan untuk melihat elemen paling atas stack tanpa menghapusnya.
Karena setelah pop() elemen yang tersisa di posisi teratas adalah **20**, maka output yang ditampilkan adalah:

Elemen teratas adalah: 20

Pada tahap ini, isi stack tetap:

- 10
- 20

5. Menampilkan Seluruh Isi Stack

System.out.println("Elemen pada stack: ");
s.print();

Bagian ini digunakan untuk menampilkan seluruh isi stack.
Method print() pada class stackArray_2511531015 akan menampilkan elemen dari posisi atas ke bawah.

Karena isi stack sekarang adalah 10 dan 20, dengan 20 berada di atas, maka hasil tampilannya adalah:

Elemen pada stack:
20 10

Hal ini menunjukkan bahwa stack ditampilkan sesuai urutan dari elemen teratas ke elemen terbawah.

Alur Kerja Program

Secara sederhana, alur kerja program ini adalah sebagai berikut:

1. Membuat stack kosong.
2. Memasukkan tiga data: 10, 20, dan 30.
3. Menghapus elemen teratas, yaitu 30.
4. Melihat elemen teratas yang baru, yaitu 20.
5. Menampilkan isi stack yang tersisa, yaitu 20 dan 10.

Konsep yang Digunakan

Program ini menerapkan konsep **stack**, yaitu struktur data linear yang hanya dapat diakses dari satu sisi, yaitu bagian atas. Operasi yang digunakan meliputi:

- push() untuk menambah data,
- pop() untuk mengambil dan menghapus data teratas,
- peek() untuk melihat data teratas,
- print() untuk menampilkan seluruh isi stack.

Prinsip utama stack yang terlihat pada program ini adalah **LIFO (Last In First Out)**, yaitu data terakhir yang masuk akan menjadi data pertama yang keluar.

PENJELASAN KODE PROGRAM STACK POSTFIX

Program stackPostfix_2511531015 dibuat untuk menghitung hasil dari **ekspresi postfix** menggunakan struktur data **stack**. Ekspresi postfix adalah bentuk penulisan operasi matematika di mana operator ditulis **setelah** operand. Contohnya:

5 2 4 * + 7 -

Berbeda dengan bentuk biasa (infix), ekspresi postfix tidak memerlukan tanda kurung karena urutan pengerjaan sudah jelas dari posisi operand dan operatornya.

Program ini menggunakan class Stack<Integer> dari Java untuk menyimpan bilangan sementara selama proses perhitungan.

Tujuan Program

Tujuan dari program ini adalah:

- memahami penerapan stack dalam evaluasi ekspresi postfix,
- melatih penggunaan operasi push() dan pop() pada stack,
- serta menghitung hasil operasi matematika berdasarkan urutan postfix.

Fungsi Method postfixEvaluate(String expression)

Method postfixEvaluate() adalah bagian utama program yang bertugas menghitung nilai dari sebuah ekspresi postfix.

public static int postfixEvaluate(String expression)

Method ini menerima satu parameter berupa String expression, yaitu ekspresi postfix yang akan dihitung, lalu mengembalikan hasil akhirnya dalam bentuk bilangan bulat (int).

Penjelasan Setiap Bagian Kode

1. Membuat Stack dan Scanner

```
Stack<Integer> s = new Stack<Integer>();
Scanner input = new Scanner(expression);
```

- Stack<Integer> s digunakan untuk menyimpan operand sementara.
- Scanner input digunakan untuk membaca isi ekspresi satu per satu berdasarkan spasi.

Jadi, ekspresi "5 2 4 * + 7 -" akan dibaca per token:

- 5
- 2
- 4
- *
- +
- 7
- -

2. Membaca Token Satu per Satu

```
while (input.hasNext()) {
```

Perulangan ini akan terus berjalan selama masih ada token yang belum dibaca dari ekspresi.

3. Jika Token adalah Bilangan

```
if (input.hasNextInt()) {
    s.push(input.nextInt());
}
```

Jika token yang dibaca adalah bilangan bulat, maka bilangan tersebut dimasukkan ke dalam stack dengan push().

Contohnya:

- token 5 → masuk ke stack
- token 2 → masuk ke stack
- token 4 → masuk ke stack

Isi stack menjadi:

5, 2, 4

4. Jika Token adalah Operator

```
else {
   String operator = input.next();
   int operand2 = s.pop();
   int operand1 = s.pop();
```

Jika token bukan bilangan, berarti token tersebut adalah operator. Program kemudian:

1. membaca operator,
2. mengambil dua operand dari stack,
3. melakukan operasi sesuai operator,
4. lalu memasukkan hasilnya kembali ke dalam stack.

Urutan pengambilan operand sangat penting:

- operand2 diambil lebih dulu dari atas stack,
- operand1 diambil setelahnya.

Hal ini penting terutama untuk operasi pengurangan dan pembagian, karena urutannya memengaruhi hasil.

5. Melakukan Operasi Matematika

```
if(operator.equals("+")) {
   s.push(operand1 + operand2);
} else if (operator.equals("-")) {
   s.push(operand1 - operand2);
} else if (operator.equals("*")) {
   s.push(operand1 * operand2);
} else {
   s.push(operand1 / operand2);
}
```

Program memeriksa operator yang dibaca:

- jika +, maka kedua operand dijumlahkan,
- jika -, maka dilakukan pengurangan,
- jika *, maka dilakukan perkalian,
- selain itu dianggap pembagian /.

•

Cara Kerja Program pada Ekspresi yang Diuji

Ekspresi yang dihitung pada method main() adalah:

"5 2 4 * + 7 -"

Mari kita urutkan prosesnya:

Langkah 1

Token:5
Karena bilangan, maka dimasukkan ke stack.

Stack:5

Langkah 2

Token:2
Masuk ke stack.

Stack:

5, 2

Langkah 3

Token:4
Masuk ke stack.

Stack:

5, 2, 4

Langkah 4

Token:*
Ambil dua elemen teratas:

- operand2 = 4
- operand1 = 2

Hitung:

2 * 4 = 8

Masukkan hasil ke stack.

Stack:

5, 8

Langkah 5

Token:+
Ambil dua elemen teratas:

- operand2 = 8
- operand1 = 5

Hitung:

5 + 8 = 13

Masukkan hasil ke stack.

Stack:

13

Langkah 6

Token:7
Masuk ke stack.

Stack:

13, 7

Langkah 7

Token:-
Ambil dua elemen teratas:

- operand2 = 7
- operand1 = 13

Hitung:

13 - 7 = 6

Masukkan hasil ke stack.

Stack:6

Karena semua token sudah selesai diproses, isi stack terakhir adalah 6.

6. Mengembalikan Hasil

```
input.close();
return s.pop();
```

Setelah semua token selesai dibaca:

- input.close() digunakan untuk menutup scanner,
- s.pop() digunakan untuk mengambil hasil akhir dari stack.

Karena hasil akhir postfix hanya satu nilai, maka nilai itulah yang dikembalikan oleh method.

7. Method main()

```
public static void main (String[] args) {
   System.out.println("hasil postfix= " + postfixEvaluate("5 2 4 * + 7 -"));
}
```

Method main() digunakan untuk menjalankan program. Program memanggil method postfixEvaluate() dengan ekspresi postfix "5 2 4 * + 7 -", lalu menampilkan hasilnya ke layar.

Output yang dihasilkan adalah:

hasil postfix= 6

Konsep yang Digunakan

Program ini menerapkan beberapa konsep penting, yaitu:

- **stack**, untuk menyimpan operand sementara,
- **push**, untuk memasukkan operand,
- **pop**, untuk mengambil operand dan hasil,
- **scanner**, untuk membaca token dari string,
- dan **evaluasi ekspresi postfix** berdasarkan urutan operator dan operand.

Stack sangat cocok digunakan dalam evaluasi postfix karena operasi selalu melibatkan elemen-elemen terakhir yang dimasukkan.